

IFT436 - Série d'exercices #4 : les graphes

Manuel Lafond

Exercice 1: Démontrez les énoncés suivants sur les arbres.

- a. Montrez qu'un arbre avec au moins 2 sommets a au moins 2 feuilles.

Solution.

Je vous présente deux preuves. L'une est plus courte que celle faite en classe. Par contre la deuxième est une belle application de l'induction sur les sommets.

— Preuve 1 —

Soit T un arbre avec au moins deux sommets. Considérons le plus long chemin $P = (u, x_1, \dots, x_k, v)$ de T , en supposant qu'il va de u à v . Notez que u ne peut pas avoir de voisin parmi $\{x_2, x_3, \dots, v\}$ car ceci impliquerait la présence d'un cycle (la seule exception est si $P = (u, v)$, dans quel cas T a deux sommets et donc deux feuilles). Notez aussi que u ne peut pas avoir de voisin à l'extérieur de $\{x_1, x_2, \dots, x_k, v\}$ car s'il y avait un tel voisin w , alors $P' = (w, u, x_1, \dots, x_k, v)$ serait un plus long chemin que P , contre-disant notre choix. Donc, u a un seul voisin, qui est x_1 . En utilisant les mêmes arguments, on déduit que v a aussi un seul voisin. Donc u et v sont des feuilles.

— Preuve 2 —

Voici la preuve par induction faite en classe.

On montre l'énoncé par induction sur n . Cas de base: si $n = 2$, le seul arbre est $T = (\{u, v\}, \{uv\})$ et a effectivement deux feuilles. Induction: on suppose que tout arbre ayant moins de n sommets a au moins 2 feuilles. Soit $T = (V, E)$ un arbre, et soit $v \in V$ un sommet arbitraire. Considérez $T' = T[V \setminus \{v\}]$ l'arbre obtenu en retirant v et ses arêtes incidentes. Alors T' a un certain nombre de composantes connexes $T_1, \dots, T_k$, toutes des arbres car retirer un sommet ne peut pas créer de cycle.

Supposons d'abord que $k \geq 2$. Considérons T_i , $1 \leq i \leq k$. Si T_i a un seul sommet, alors ce sommet est une feuille dans T car son seul voisin est v . Si T_i a au moins deux sommets, par induction T_i a deux feuilles f_1 et f_2 . On sait que v n'a qu'un seul voisin dans T_i , sinon il y aurait un cycle dans T . Donc au moins un de f_1 ou f_2 est une feuille aussi dans T , car réinsérer v ne lui ajoute pas de voisin. On vient de montrer, en particulier, que T_1 a un sommet qui est une feuille dans T et T_2 aussi. Donc, T a au moins deux feuilles.

Supposons plutôt que $k = 1$. Par les mêmes arguments que ci-haut, T_1 a un sommet f qui est une feuille dans T . Par ailleurs, v est une feuille dans T puisqu'il a un seul voisin dans T_1 (et $k = 1$). Il s'ensuit que f et v sont deux feuilles de T .

- b. Montrez que si un arbre T a un sommet de degré k , alors T a au moins k feuilles.

Solution.

Soit v un sommet de degré k avec les voisins $u_1, \dots, u_k$. Considérons $T' = T[V \setminus \{v\}]$. Soient $T_1, \dots, T_h$ les composantes connexes de T' . Il est facile de voir que pour tout u_i, u_j distincts, u_i et u_j ne sont pas dans la même composante: s'il y avait un chemin entre u_i et u_j dans T' , l'ajout de v à ce chemin créerait un cycle dans T .

Donc en fait, T' est constitué d'au moins k arbres $T_1, \dots, T_k$. En réutilisant les arguments de l'induction de la preuve précédente, on peut déduire que chaque T_i a au moins un sommet qui est une feuille dans T . On déduit que T a au moins k feuilles.

- c. Montrez aussi que le seul arbre à n sommets deux feuilles est un chemin de longueur $n - 1$.

Solution.

Si un arbre T n'est pas un chemin, il doit avoir un sommet de degré au moins 3, car la seule façon de n'avoir que des sommets de degré 1 et 2 est d'avoir un chemin (idéalement, il faudrait démontrer, peut-être par induction, mais nous ne nous étendrons pas).

Par le résultat précédent, T a donc au moins 3 feuilles. Donc tout arbre qui n'est pas un chemin a au moins 3 feuilles.

Exercice 2: Donnez le code d'un algorithme qui trouve le chemin le plus court d'un sommet u à un sommet v sur un graphe *orienté*.

Indice: c'est presque trivial.

Solution.

Reprenez l'algorithme pour le cas non-orienté, et assurez-vous d'aller vers les voisins sortants à chaque itération.

Exercice 3: Donnez un algorithme qui détermine un si graphe G est un arbre.

Solution.

Le plus simple est d'utiliser la définition disant que G est un arbre s'il est connexe et a $n - 1$ arêtes. L'algorithme va comme suit:

- si $\text{composantesConnexes}(G) > 1$, return false
- si $|E| \neq n - 1$, return false
- return true

Exercice 4: Montrez que si un graphe G a k arêtes, avec $k < n$, alors G a au moins $n - k$ composantes connexes.

Solution.

Voir théorème 8, notes de cours.

Exercice 5: Une stratégie naïve pour trouver le chemin le plus court entre deux sommets u et v est d'énumérer *tous* les chemins, et retourner le plus court.

C'est une mauvaise idée.

Pour le voir, montrez qu'il existe au moins $(n-2)!$ chemins entre deux sommets u et v dans un graphe complet.

Notez que le graphe complet a $\Theta(n^2)$ arêtes. Peut-être que sur les graphes avec peu d'arêtes, il y a peu de chemins. Montrez que c'est faux. C'est-à-dire, montrez qu'il existe des graphes avec $O(n)$ arêtes mais $\Omega(2^n)$ chemins entre deux sommets u et v .

Solution.

Dans un chemin entre u et v d'un graphe complet, il y a $n - 2$ choix pour le premier sommet à visiter à partir de u , puis $n - 3$ choix pour le prochain sommet, et ainsi de suite jusqu'à ce qu'il ne reste qu'un choix pour le prédécesseur de v (à chaque étape, tous les choix restants sont possibles parce que le graphe est complet). Le nombre de chemins est donc $(n - 2)(n - 3) \dots (2)(1) = (n - 2)!$.

Pour la deuxième partie, il faut faire attention de bien avoir $\Omega(2^n)$ sommets et non quelque chose du style $\Omega(2^{n/2})$, ce qui n'est pas la même chose. Considérez le graphe $G = (V, E)$ avec $V = \{a_1, a'_1, a_2, a'_2, \dots, a_k, a'_k\}$ et, pour chaque $i \in \{1, \dots, k - 1\}$, les arêtes $a_i a'_i, a_i a_{i+1}, a_i a'_{i+1}, a'_i a_{i+1}$ et $a'_i a'_{i+1}$. Je vous invite à dessiner le graphe avec $k = 3$ pour le visualiser.

Ce graphe a $n = 2k$ sommets et chaque sommet a au maximum 5 voisins. Donc le nombre d'arêtes est au maximum $10k \in \Omega(n)$, car ça diffère de n à une constante près. Comptons le nombre de façons de passer d'un sommet a_i à un sommet a_{i+1} ou bien a'_{i+1} , en supposant que a'_i n'a pas été visité. On peut soit passer de a_i à a_{i+1} ou a'_{i+1} directement, ou bien aller à a'_i pour ensuite aller à a_{i+1} ou a'_{i+1} . Il y a donc 4 façons. Le même argument s'applique si on veut passer de a'_i à la couche $i + 1$. Maintenant, considérez le nombre de chemins qui démarrent de a_1 ou bien a'_1 et qui se rendent à a_k ou a'_k . Il y a 4 façons d'aller de la couche a_1 à a_2 , puis 4 façons d'aller de la couche a_2 à a_3 , et ainsi de suite. Le nombre de changements de couche est $k - 1$, et donc ceci nous fait au moins $4^{k-1} = 4^k / 4 = 4^{n/2} / 4 = 2^n / 4 \in \Omega(2^n)$.

Exercice 6: Le *diamètre* d'un graphe est la plus longue distance à parcourir pour aller d'un sommet à un autre (en supposant que l'on prend le chemin le plus court). Formellement, le diamètre s'écrit $diam(G)$ et se définit par

$$diam(G) = \max_{u,v \in V} dist(u, v)$$

où $dist(u, v)$ est la longueur du chemin le plus court entre u et v .

Donnez un algorithme en temps $O(n^2 + nm)$ qui trouve le diamètre d'un graphe.

Solution.

Il n'est pas difficile de modifier la fouille en largeur pour obtenir, pour $u \in V$, le sommet le plus éloigné de u . Pour trouver le diamètre, on fait une fouille en largeur pour chaque $u \in V$, et le diamètre sera le maximum de la distance entre un u et son sommet le plus éloigné. Ceci prendra un temps $O(n(n + m)) = O(n^2 + nm)$.

Exercice 7: Comparez la complexité des opérations “Supprimer une arête” et “Supprimer un sommet” sur une liste d’adjacence et sur une matrice d’adjacence. Notez que lorsqu’on supprime un sommet, toutes ses arêtes incidentes sont supprimées.

Solution. Sur une liste d’adjacence, supprimer une arête uv demande de retirer un élément des deux listes pour u et v , ce qui demande un temps $O(deg(u) + deg(v))$. Pour supprimer un sommet v , il faut passer chaque liste dans laquelle v est présent. Ceci demande de parcourir chaque liste pour voir si v y est et, si oui, d’y faire une suppression. On peut optimiser un peu, mais il est difficile d’argumenter que c’est mieux que $O(n + \sum_{u \in V} deg(u)) \in O(m)$.

Sur une matrice d’adjacence, retirer une arête se fait en temps $O(1)$ mais retirer un sommet demande de recréer toute la matrice en temps $O(n^2)$.

Exercice 8: Dans un graphe connexe G , un *point d’articulation* est un sommet v tel que $G[V \setminus \{v\}]$ n’est pas connexe. Donnez un algorithme qui trouve tous les points d’articulation d’un graphe.

Indice: ce n’est pas une question immédiatement évidente. Considérez l’arbre de fouille en profondeur.

Solution. Considérez l’arbre de fouille en profondeur démarré avec un sommet u . Démontrez qu’un sommet v est un point d’articulation si et seulement si $v = u$ et u a au moins 2 enfants sur l’arbre de la fouille, ou si $v \neq u$ et que pour tout descendant w de v dans cet arbre, w n’a aucun voisin parmi les ancêtres de v . une fois que vous avez ces propriétés, elles se vérifient rapidement pendant une fouille en profondeur.

Exercice 9: Donnez un algorithme qui trouve toutes les composantes fortement connexes d'un graphe orienté. Les composantes fortement connexes sont définies dans le devoir 3.

Solution. Voir l'algorithme décrit ici: https://fr.wikipedia.org/wiki/Algorithme_de_Kosaraju.

Exercice 10: Démontrez qu'un graphe orienté G avec n sommets a n composantes fortement connexes si et seulement si G admet un tri topologique.

Solution. Supposons qu'il y a n composantes fortement connexes. Il ne peut pas y avoir de cycle orienté, car deux sommets dans un même cycle orienté doivent être dans une même composante. Ceci implique qu'il y a un tri topologique. Dans l'autre sens, supposons que G admet un tri topologique $(v_1, \dots, v_n)$. On peut le montrer par induction sur n , le cas de base $n = 1$ étant trivial. Pour un n plus grand, on voit que v_n forme une composante fortement connexe en soi car il ne peut aller nulle part. En se rappelant que les composantes fortement connexes forment une partition de V , on peut retirer v_n pour obtenir les autres composantes fortement connexes de G . Si on retire v_n , on sait par induction que le graphe résultant a $n - 1$ composantes fortement connexes (car le graphe admet aussi un tri topologique). Ceci démontre l'énoncé.